

ECOM EXPERTS

Ecom blockbot

Manual de usuario, tecnico y de desarrollo

ecom_blockbot 1.0.0 · 08/09/2026

Contenido

Qué resuelve

Cómo lo consigue sin captcha

Puesta en marcha

La pantalla de configuración

General

Formularios

Comprobaciones

Contenido y correo

Límites y bloqueos

La pantalla del registro

Preguntas que nos hacen siempre

1. Qué hace

2. Cómo lo hace: la idea en una frase

El mecanismo, paso a paso

Las comprobaciones

Cómo se puntúa

Lo que se ve por dentro

3. Cómo saber si de verdad está bloqueando

Con el botón (lo más fácil)

A mano, desde cualquier terminal

Los dos avisos de la pantalla

4. El registro: qué se ha bloqueado y por qué

5. La configuración, pestaña por pestaña

General

Formularios

Comprobaciones

Contenido y correo

Límites y bloqueos

6. Preguntas frecuentes

7. Lo que este módulo NO hace

8. Si tu tienda está muy personalizada

1. Qué hace y por qué así

La idea central: nada se pinta en el HTML

Flujo de una persona

2. Dónde se corta cada formulario

Rendimiento

3. La ficha firmada y la prueba de trabajo

4. Motor de puntuación

5. Ficheros

6. Base de datos

7. Configuración y multitienda

Los nombres de los campos

8. El endpoint de la ficha

9. Pruebas

10. Límites conocidos

Reglas de la casa que este módulo cumple

Las cinco trampas que ya costaron tiempo aquí

1. Module::trans() es protected

2. Las claves de las pestañas se convierten en id del DOM

3. actionSubmitAccountBefore tiene que devolver true SIEMPRE

4. El endpoint AJAX, sin Content-Type

5. El registro no es un ObjectModel a propósito

Puntos de extensión

Proteger otro formulario

Cambiar los pesos

Añadir una comprobación

Tocar el JavaScript

Cómo se prueba

Traducciones

Depuración

Bloquea los robots en los formularios de tu tienda, sin captcha. Versión 1.0.0 · PrestaShop 8 y 9 · Ecom Experts

Qué resuelve

Si te llegan altas de cuenta falsas, mensajes de contacto con enlaces raros o suscripciones al boletín de correos que no existen, es que un programa está rellenando tus formularios de forma automática. Este módulo lo corta.

Tus clientes no ven nada. No hay imágenes borrosas, no hay «marca las motos», no hay casilla de «no soy un robot». El cliente rellena el formulario y lo envía como siempre; el módulo trabaja por detrás.

Formularios protegidos:

- **Registro de cuenta**
 - **Contacto**
 - **Suscripción al boletín**
 - **Creación de cuenta durante la compra**
 - **Identificación** (apagado de fábrica; enciéndelo si te atacan las contraseñas)
-

Cómo lo consigue sin captcha

No hay una sola defensa, hay una docena de comprobaciones invisibles. Ninguna es infalible; juntas hacen que enviar spam a tu tienda deje de salir a cuenta.

	Qué caza
Campos trampa	Un campo invisible para la persona. Si llega relleno, lo ha relleno un robot
Prueba de trabajo	El navegador tiene que resolver un cálculo antes de enviar. A tu cliente le cuesta 50 milésimas de segundo; a quien manda 100.000 envíos le cuesta electricidad y dinero
Exigir un navegador de verdad	Los programas de línea de órdenes (los más habituales) no llegan ni a conseguir el permiso para enviar
Ficha con caducidad y un solo uso	No se puede enviar el formulario directamente sin pasar por la página, ni repetir un envío que ya funcionó
Tiempo mínimo	Una persona tarda segundos en rellenar; un robot envía al instante
Señales de comportamiento	Ratón, teclado, toques... y la marca que dejan los navegadores automatizados
Análisis del navegador	Detecta a los que dicen ser <code>curl</code> , Python o un navegador sin pantalla
Límite por dirección IP	Nadie se registra ocho veces en una hora
Correos de usar y tirar	Casi 600 dominios de correo temporal, y puedes añadir más
Dominio que no recibe correo	Si el dominio del correo no puede recibir nada, no es un cliente
Análisis del mensaje	Enlaces de más, alfabetos que no pintan nada en tu tienda, palabras que tú marques
Bloqueo automático	Quien insiste se queda fuera unas horas, y se le desbloquea solo

Puesta en marcha

1. Instala el módulo.
2. Abre **Módulos** → **Ecom Blockbot** → **Configurar**.
3. Comprueba que arriba pone, en verde, **«El punto de entrega de fichas responde bien. La protección está funcionando.»**

Ya está. Viene configurado y protegiendo desde el primer momento, con valores prudentes: de fábrica solo rechaza lo que es indudable.

Si ese aviso sale en rojo, la protección no está funcionando y, mientras siga así, ningún visitante podrá enviar un formulario protegido. Suele ser cosa de las URL amigables o del `.htaccess`. Apaga el módulo (Modo → Apagado) mientras lo arreglas.

La pantalla de configuración

Todo está en una sola pantalla con cinco pestañas y **un único botón de Guardar**.

General

- **Modo**
- *Apagado*: el módulo no hace absolutamente nada.
- *Aprendizaje*: lo registra todo pero **no bloquea nada**. Úsalo unos días si quieres comprobar con tus propios ojos que no está cayendo ningún cliente de verdad.
- *Protección*: bloquea. Es el modo de fábrica.
- **Umbral de bloqueo**: los puntos que hacen falta para rechazar un envío. Con **100** (de fábrica) solo bloquean solas las comprobaciones que no se equivocan nunca. Bajarlo a 70 aprieta bastante más, pero puede rechazar a alguien de verdad: hazlo solo si sigues recibiendo spam y después de mirar el registro.
- **Mensaje que ve el visitante bloqueado**: déjalo vacío y se usará el del módulo, en el idioma de quien lo lee.
- **Días que se conserva el registro y registrar también los envíos aceptados** (esto último hace falta para que los límites por IP cuenten bien).
- **Modo de depuración**: solo si te lo pedimos para investigar algo.

Formularios

Un interruptor por formulario. Si tienes un formulario de contacto de otro módulo, es posible que no esté cubierto: pruébalo.

Comprobaciones

- **Exigir una ficha firmada (JavaScript)**: es la comprobación más importante y la que corta el 95 % del spam. El único coste es que **un visitante con JavaScript desactivado no podrá enviar** (hoy es menos del 0,2 % de la gente, y esa gente ya no puede usar medio PrestaShop).
- **Dificultad de la prueba de trabajo**: 30.000 cuesta unos 40 ms en el navegador. Subirlo le cuesta más al robot **y también a tu cliente**; no lo toques sin motivo.
- **Segundos mínimos antes de enviar**: 2 de fábrica. Súbelo a 4 o 5 si te llega spam que pasa el resto de filtros.
- **Minutos que vale la ficha**: 30. Si algún cliente deja el formulario abierto media hora, el módulo pide una ficha nueva solo, sin que se entere.

Contenido y correo

Correos de usar y tirar, comprobación de que el dominio recibe correo, número de enlaces permitidos en un mensaje, palabras vigiladas (una por línea) y alfabetos que no pintan nada en tu tienda.

Si vendes a Rusia, China o países árabes, **desmarca esos alfabetos**: si no, estarás penalizando a clientes de verdad.

Límites y bloqueos

Envíos permitidos por IP y hora / por IP y día, y el bloqueo automático: cuántas detecciones hacen falta y cuántas horas dura. Un ☐ desactiva el límite correspondiente.

La pantalla del registro

Cientes → **Ecom Blockbot**. Ahí está todo lo que ha pasado.

Arriba, las **listas blanca y negra de IP**. Puedes escribir una dirección suelta (`88.20.30.40`), un rango (`88.20.30.0/24`) o un comodín (`88.20.30.*`), decir si va a la lista blanca o a la negra, cuántas horas dura (0 = para siempre) y por qué.

La lista blanca siempre gana. A una IP que esté ahí no se le comprueba absolutamente nada. Es la salida de emergencia si alguna vez bloqueas a alguien por error.

Debajo, el **registro de detecciones**: fecha, formulario, decisión, puntuación, los motivos en cristiano, la IP, el correo y el navegador declarado. Se puede filtrar por cualquiera de esas columnas. Cada línea trae dos botones, **Bloquear** y **Permitir**, para meter esa IP en una lista de un clic.

Preguntas que nos hacen siempre

Un cliente me llama y dice que no puede registrarse. Pregúntale su IP (en `cualesmiip.com`), búscala en el registro y mira el motivo. Si ha sido un error, mete esa IP en la lista blanca desde la propia línea y listo. Si el motivo es «Sin ficha de validación», casi seguro que tiene JavaScript desactivado o una extensión que lo bloquea.

¿Ralentiza la tienda? No. En una página normal el módulo no hace ni una consulta a la base de datos. Solo trabaja cuando alguien envía uno de los formularios protegidos.

¿Envía datos a algún sitio? No. Todo se resuelve dentro de tu tienda: no hay ningún servicio externo, ni Google, ni ninguna clave de API que pedir.

¿Funciona con mi plantilla? Sí. El módulo no toca ninguna plantilla ni instala ningún *override*; localiza los formularios por dentro, de una forma que no depende del diseño.

Tengo un módulo de caché (Varnish, un CDN...). No hay problema: la parte que cambia en cada visita se pide aparte y no se cachea nunca.

Tengo varias tiendas. Toda la configuración es de la tienda que tengas seleccionada arriba, menos la clave de firma y el prefijo de los campos, que son comunes. La pantalla te lo recuerda.

Sigo recibiendo spam. Por este orden: 1) mira el registro, a ver si esos envíos aparecen y con qué puntuación; 2) sube los **segundos mínimos** a 4 o 5; 3) baja el **umbral** a 70; 4) añade las palabras que se repitan a la lista de vigilancia; 5) baja **detecciones antes de bloquear una IP** a 2.

He desactivado el módulo y quiero volver a activarlo. No pierdes nada: la configuración, el registro y las listas de IP siguen ahí. Solo al **desinstalarlo** se borran.

Ecom Blockbot — manual

Qué hace, cómo lo hace y cómo comprobar que está funcionando. Versión 1.0.0 · PrestaShop 8 y 9 · Ecom Experts · ecomyseo@gmail.com

La versión ilustrada y autocontenida de este manual es `Manual-Ecom-Blockbot.pdf` , aquí al lado. En este .md las imágenes solo se ven desde la copia de trabajo, porque las capturas viven fuera del módulo (en `modulos IA\capturas\ecom_blockbot\8.2b\`) para no acabar dentro del ZIP del cliente.

1. Qué hace

Rechaza los envíos **automáticos** de estos formularios:

- Registro de cuenta
- Contacto
- Suscripción al boletín
- Creación de cuenta durante la compra
- Identificación (*apagado de fábrica*)

Sin captcha de ningún tipo. Tu cliente no ve nada nuevo: ni imágenes, ni «marca los semáforos», ni la casilla de «no soy un robot». Rellena el formulario y lo envía como siempre.

2. Cómo lo hace: la idea en una frase

No pone una prueba delante del cliente. Pone una veintena de comprobaciones invisibles detrás.

Ninguna es infalible por separado. Juntas hacen que mandar spam a tu tienda deje de salir a cuenta, que es de lo que se trata: el spam es un negocio, y si cuesta más de lo que rinde, se van a otra tienda.

El mecanismo, paso a paso

Esto es lo que pasa cuando una **persona** entra en tu página de registro:

- | | |
|--|--|
| 1. Abre la página | El HTML que sale del servidor NO lleva nada del módulo. Se puede cachear tranquilamente. |
| 2. Su navegador ejecuta el JavaScript del módulo | Localiza el formulario y le mete, en el sitio: <ul style="list-style-type: none">• 2 campos TRAMPA, invisibles• 3 campos ocultos vacíos |
| 3. Toca el formulario (pincha en un campo, mueve el ratón, teclea) | El JavaScript pide al servidor una FICHA firmada |
| 4. El servidor le manda un RETO | Un cálculo que hay que resolver por fuerza bruta |
| 5. El navegador lo resuelve | Le cuesta unas 50 milésimas de segundo |
| 6. La persona pulsa Enviar | Los campos ocultos ya van rellenos |
| 7. El servidor comprueba | Ficha válida, no usada, no caducada, cálculo resuelto, campos trampa vacíos... y deja pasar |

Y esto es lo que le pasa a un **robot** de los que mandan spam (un script de `curl`, Python o similar, que es la inmensa mayoría):

- | | |
|---|--|
| 1. Envía el formulario directamente a la URL, sin abrir la página | |
| 2. FIN. | No ha ejecutado JavaScript, así que no tiene ficha. Sin ficha no hay envío. Se queda en el paso 1. |

Las comprobaciones

	Qué caza	Por qué funciona
Campos trampa	Robots que rellenan todo lo que encuentran	Son invisibles para la persona. Si llegan rellenos, los ha rellenado un programa
Ficha firmada	Envíos directos a la URL	La ficha la firma el servidor con una clave secreta. No se puede falsificar
Un solo uso	Repetir un envío que funcionó	Cada ficha se anota al gastarse. La segunda vez se rechaza
Caducidad	Fichas guardadas para más tarde	A los 30 minutos deja de valer
Prueba de trabajo	Envíos masivos	El navegador tiene que resolver un cálculo. A tu cliente le cuesta 50 ms; a quien manda 100.000 envíos le cuesta CPU, tiempo y dinero
Exigir JavaScript	Robots de línea de órdenes	Es la comprobación que corta el 95 % del spam
Tiempo mínimo	Robots rápidos	Una persona tarda segundos en rellenar; un robot envía al instante
Señales de comportamiento	Navegadores automatizados	Ratón, teclado, toques... y la marca <code>navigator.webdriver</code> que encienden Selenium, Puppeteer y Playwright
Análisis del navegador	Clientes que no son navegadores	Agente de usuario vacío, o que dice ser <code>curl</code> , <code>python-requests</code> , <code>HeadlessChrome</code> ...
Origen y referente	Envíos desde otro dominio	Un formulario de tu tienda se envía desde tu tienda
Límite por IP	Insistencia	Nadie se registra ocho veces en una hora
Correo desechable / sin buzón	Altas falsas	569 dominios de usar y tirar, más la comprobación de que el dominio puede recibir correo
Análisis del contenido	Spam con enlaces	Enlaces de más, alfabetos que no pintan nada en tu tienda, palabras que tú marques
Bloqueo automático	Reincidentes	Quien acumula detecciones se queda fuera unas horas, y se le desbloquea solo

Cómo se puntúa

Cada comprobación que falla **suma puntos**. Cuando se pasa del **umbral** (100 de fábrica), el envío se rechaza.

De fábrica solo llegan a 100 **por sí solas** las comprobaciones que no se equivocan nunca: campo trampa relleno, ficha ausente, ficha manipulada, cálculo sin resolver, ficha reutilizada, IP bloqueada y límite por IP. Las demás suman entre 25 y 80: hacen falta dos para rechazar. **Por eso el módulo no echa a clientes de verdad de serie.**

Lo que se ve por dentro

En esta captura se han hecho visibles a la fuerza los campos que mete el JavaScript, para que se entienda. **En tu tienda el cliente no ve ninguno de estos recuadros.**

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/manual-campos-inyectados.png] />

- En **rojo**, los dos campos trampa. Están fuera de la pantalla, sin tabulación y con `aria-hidden`, así que ni se ven ni los lee un lector de pantalla. Un robot que rellene todos los campos de texto cae aquí.
- En **verde**, los campos ocultos: la ficha firmada (con el cálculo ya resuelto) y las señales de comportamiento.

3. Cómo saber si de verdad está bloqueando

Es la pregunta importante, y el módulo la contesta él solo.

Con el botón (lo más fácil)

Módulos → **Ecom Blockbot** → **Configurar**, y abajo del todo: **«Comprobar la protección ahora»**.

Tu navegador envía los tres formularios exactamente como los enviaría un robot, y el servidor mira qué ha pasado y **borra todo lo que la prueba haya dejado**.

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/manual-autocomprobacion.png] />

Si sale **verde**, está bloqueando: en la tabla verás cada formulario con «Bloqueado» y el motivo («Sin ficha de validación»).

Si sale **rojo**, te dice exactamente qué formulario ha pasado el filtro.

Si sale **naranja**, es que a algún formulario no se ha podido llegar: eso **no** quiere decir que esté desprotegido, solo que la prueba no ha llegado. La tabla te dice de cuál se trata y por qué (la página no abre, el formulario no está en ella, o algo se ha comido el envío por el camino).

A mano, desde cualquier terminal

En la misma pantalla tienes la orden preparada con la URL de tu tienda. Es esta forma:

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST "https://TUTTIENDA.com/registro" \
-d
"submitCreate=1&firstname=Robot&lastname=Prueba&email=robot@ejemplo.test&password=Prueba1234"
```

Después, en **Clientes → Ecom Blockbot**, tiene que haber aparecido una línea nueva: **Bloqueado · Sin ficha de validación**.

Si no aparece nada, el módulo no está interceptando ese formulario.

Los dos avisos de la pantalla

Arriba del todo hay dos semáforos que se comprueban solos cada vez que abres la pantalla:

1. «**Modo protección: se están rechazando los envíos automatizados**» — el módulo está encendido y bloqueando.
2. «**El punto de entrega de fichas responde bien**» — la parte que da la ficha a los clientes está viva.

⚠ Si el segundo sale **en rojo**, ningún visitante puede enviar un formulario protegido: apaga el módulo (Modo → Apagado) mientras lo arreglas. Suele ser cosa de las URL amigables o del `.htaccess`.

4. El registro: qué se ha bloqueado y por qué

Clientes → Ecom Blockbot

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/manual-registro.png] />

Cada línea es un envío analizado: fecha, formulario, decisión, puntuación, **los motivos en cristiano**, la IP, el correo y el navegador que decía ser.

Se puede filtrar por cualquier columna. Y cada línea trae dos botones, **Bloquear** y **Permitir**, para meter esa IP en una lista de un clic.

Arriba están las **listas blanca y negra**. Admiten una IP suelta (`88.20.30.40`), un rango (`88.20.30.0/24`) o un comodín (`88.20.30.*`), con caducidad opcional.

La lista blanca siempre gana. A una IP que esté ahí no se le comprueba nada. Es la salida de emergencia si alguna vez bloqueas a alguien por error.

5. La configuración, pestaña por pestaña

Una sola pantalla, cinco pestañas y **un único botón de Guardar**.

General

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/config-bbgeneral.png] />

- **Modo:** *Apagado / Aprendizaje* (registra pero no bloquea) / *Protección*.

- **Umbral de bloqueo:** 100 de fábrica. Bajarlo a 70 aprieta mucho más, pero puede rechazar a alguien de verdad. Mira antes el registro.
- **Mensaje que ve el visitante bloqueado:** vacío = el del módulo, en su idioma.
- **Días que se conserva el registro y registrar también los envíos aceptados.**
- **Modo de depuración:** escribe un `.log` dentro de la carpeta del módulo.

Formularios

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/config-bbforms.png] />

Un interruptor por formulario.

Comprobaciones

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/config-bbchecks.png] />

- **Exigir una ficha firmada (JavaScript):** la comprobación clave. Un visitante con JavaScript desactivado no podrá enviar (hoy es menos del 0,2 % de la gente).
- **Dificultad de la prueba de trabajo:** 30.000 $\approx$ 40 ms. Subirlo le cuesta más al robot **y también a tu cliente**.
- **Segundos mínimos antes de enviar:** 2. Súbelo a 4 o 5 si te sigue entrando spam.
- **Minutos que vale la ficha:** 30. Si un cliente deja el formulario abierto más tiempo, el módulo pide una ficha nueva solo, sin que se entere.

Contenido y correo

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/config-bbcontent.png] />

Correos desechables, comprobación de que el dominio recibe correo, enlaces permitidos, palabras vigiladas y alfabetos ajenos.

Si vendes a Rusia, China o países árabes, **desmarca esos alfabetos**.

Límites y bloqueos

[imagen no encontrada: ../capturas/ecom_blockbot/8.2b/config-bblimits.png] />

Envíos por IP y hora / por IP y día, y el bloqueo automático de reincidentes. Un `0` desactiva el límite correspondiente.

6. Preguntas frecuentes

Un cliente dice que no puede registrarse. Pídele su IP (en `cualesmiip.com`), búscala en el registro y mira el motivo. Si fue un error, mete la IP en la lista blanca desde la propia línea. Si el motivo es «Sin ficha de validación», casi seguro que tiene JavaScript desactivado o una extensión que lo bloquea.

¿Ralentiza la tienda? No. En una página normal el módulo no hace ni una consulta a la base de datos: solo trabaja cuando alguien envía uno de los formularios protegidos.

¿Manda datos a algún sitio? No. Todo se resuelve dentro de tu tienda. Ni Google, ni servicios externos, ni claves de API que pedir. Nada que declarar en la política de privacidad por este módulo.

¿Funciona con mi plantilla? Sí. No toca ninguna plantilla ni instala ningún *override*: localiza los formularios por el nombre del campo de envío, que lo pone PrestaShop y no el tema.

Tengo caché de página completa (Varnish, un CDN...). Sin problema: lo único que cambia en cada visita se pide aparte y nunca se cachea.

Tengo varias tiendas. Toda la configuración es de la tienda que tengas seleccionada arriba, menos la clave de firma y el prefijo de los campos, que son comunes. La pantalla te lo recuerda.

Sigo recibiendo spam. Por este orden: 1) mira el registro, a ver si esos envíos aparecen y con qué puntuación; 2) sube los **segundos mínimos** a 4 o 5; 3) baja el **umbral** a 70; 4) añade a las palabras vigiladas lo que se repita; 5) baja **detecciones antes de bloquear una IP** a 2.

¿Y si lo desactivo? No pierdes nada: configuración, registro y listas siguen ahí. Solo al **desinstalarlo** se borra todo.

7. Lo que este módulo NO hace

Para que no haya sorpresas:

- **No es impenetrable.** Un atacante decidido, con un navegador real automatizado y bien disfrazado, puede pasar. El objetivo es que el spam masivo deje de salir a cuenta, no detener a alguien que va específicamente a por tu tienda.
- **No protege los comentarios de producto** ni formularios de módulos de terceros que no usen uno de los campos de envío estándar de PrestaShop.
- **No filtra el correo que sale** de la tienda ni el contenido de los pedidos.
- **No bloquea el rastreo del catálogo:** esto va de envíos de formulario, no de lectura.
- **No protege el back-office:** para eso PrestaShop tiene su propio 2FA.

8. Si tu tienda está muy personalizada

El registro y el boletín se cortan en dos sitios a la vez: en el despachador de PrestaShop —antes de que la petición llegue a ningún controlador— y en el hook correspondiente. No es redundancia por gusto.

Muchos temas y módulos de compra en un paso traen un **override del controlador de registro** que no llama a los hooks del núcleo. Y el hook del boletín solo lo dispara el módulo oficial `ps_emailsubscription`: con otro módulo de boletín, ese hook no existe. En los dos casos, un módulo que solo use el hook deja el formulario sin proteger **y nadie se entera**, porque no falla nada: el envío simplemente pasa. Cortando antes, en el despachador, no hay override ni módulo de terceros que pueda saltárselo.

Si tu tienda tiene overrides o un tema muy tocado, el botón «**Comprobar la protección ahora**» es justo lo que hay que pulsar después de cada cambio de tema o de módulo.

Ecom Blockbot — documentación técnica

Versión: 1.0.0 **Compatibilidad:** PrestaShop 8.0 – 9.x (estructura *legacy*, sin `src/`, sin Composer) **Autor:** Ecom Experts · ecomyseo@gmail.com · licencia AFL-3.0

1. Qué hace y por qué así

Rechaza los envíos automatizados de los formularios de **registro de cuenta, contacto, suscripción al boletín y creación de cuenta durante la compra**, sin mostrar ningún captcha al visitante.

En vez de una prueba visible, acumula comprobaciones invisibles y baratas. Ninguna es infalible por separado; juntas hacen que automatizar el envío deje de salir a cuenta.

La idea central: nada se pinta en el HTML

El módulo **no toca ni una plantilla** —ni del tema ni del núcleo— y **no usa overrides**. Sus campos los inyecta **JavaScript en el navegador**. Eso resuelve tres cosas a la vez:

1. funciona con cualquier tema, porque se localiza el formulario por el nombre del campo de envío, que lo pone PrestaShop y no el tema;
2. sobrevive a la caché de página completa, porque la ficha de validación no viaja nunca dentro del HTML;
3. **el simple hecho de que los campos existan ya demuestra que hay un navegador de verdad ejecutando código.**

Flujo de una persona

- | | |
|--|--|
| 1. Abre la página del formulario | HTML cacheable, sin nada del módulo |
| 2. El JS localiza el formulario y le inyecta 2 campos trampa + 3 ocultos | |
| 3. La persona lo toca (foco, tecla, ratón) → el JS pide la ficha | POST /module/ecom_blockbot/challenge
(no cacheable, reto firmado) |
| 4. Resuelve la prueba de trabajo en 40-100 ms y rellena los ocultos | |
| 5. Envía. El servidor comprueba y pasa | |

Un robot de línea de órdenes se queda en el paso 1: no hay campos, no hay ficha, no pasa.

2. Dónde se corta cada formulario

Formulario	Cómo se detecta	Hook	Cómo se avisa
Registro de cuenta	<code>submitCreate</code> y <code>php_self != order</code>	<code>actionDispatcher</code> → se quita <code>submitCreate</code> del POST, y además <code>actionSubmitAccountBefore</code>	<code>\$this->context->controller->errors[]</code>
Registro en la compra	<code>submitCreate</code> y <code>php_self = order</code>	los dos, igual	igual
Contacto	<code>submitMessage</code> en el POST	<code>actionDispatcher</code> → se quita <code>submitMessage</code> del POST	<code>\$this->context->controller->errors[]</code>
Boletín	<code>submitNewsletter</code> en el POST	<code>actionDispatcher</code> → se quita <code>submitNewsletter</code> , y además <code>actionNewsletterRegistrationBefore</code>	<code>\$this->context->controller->errors[]</code>
Identificación (apagado de fábrica)	<code>submitLogin</code> en el POST	<code>actionDispatcher</code> → se quita <code>submitLogin</code>	<code>\$this->context->controller->errors[]</code>

Verificado contra el código real de PrestaShop:

- `Dispatcher.php:507` (8.2.7) y `:465` (9.1.1) → `Hook::exec('actionDispatcher', ...)` justo antes de `$controller->run()`, con el controlador ya en `Context::getContext()->controller`.
- `RegistrationController.php:56` y `CheckoutPersonalInformationStep.php:74` → `array_reduce(Hook::exec('actionSubmitAccountBefore', ...))`: si un módulo devuelve `false`, no se crea la cuenta. **Por eso el hook devuelve siempre true explícitamente:** devolver `null` bloquearía todos los registros de la tienda.

El registro y el boletín se cortan en DOS sitios a propósito. Ese hook lo dispara el controlador del núcleo, y basta con que la tienda tenga un `override/controllers/front/RegistrationController.php` —los traen muchos temas y módulos de compra en un paso— para que **no se ejecute nunca:** el formulario se quedaría sin proteger y nadie se enteraría. Por eso se corta también en `actionDispatcher`, que es anterior al controlador y no lo puede saltar ningún override. Comprobado el 02/09/2026 en dos tiendas de cliente: sin este corte, el registro pasaba en una tienda con override del controlador, y la suscripción al boletín pasaba en otra que no usa `ps_emailsubscription` (ese hook solo lo dispara el módulo oficial). `EcomBlockbotDetector::analyze()` cachea el resultado por formulario, así que el doble corte no analiza ni consume la ficha dos veces. - `ps_emailsubscription.php:424` → `actionNewsletterRegistrationBefore` CON `'hookError' => &$hookError`. - `contactform.php:253` → `getWidgetVariables()` llama a `sendMessage()` si `Tools::isSubmit('submitMessage')`. **No hay ningún hook antes**, de ahí el corte en `actionDispatcher`. - `FrontController.php:750` → `'notifications' => $this->prepareNotifications()`, que sale de `$this->errors`.

Rendimiento

`hookActionDispatcher` sale en la primera línea si no hay POST, si el módulo está apagado o si el POST no trae ninguna de las claves vigiladas: **cero consultas** en la inmensa mayoría de las peticiones.

`Configuration::get` no consulta la base de datos (PrestaShop la carga entera al arrancar).

El JavaScript son ~19 KB y se encola con `defer` en el pie, solo si hay al menos un formulario protegido activo.

3. La ficha firmada y la prueba de trabajo

Diseño tipo ALTCHA, entero dentro del módulo. **Sin sesión de PHP** (nada de bloqueo de sesión) y **sin llamadas a terceros**.

El servidor emite (`EcomBlockbotChallenge::issue`):

```
number    = aleatorio en [0, maxnumber]          NO se envía
salt      = 16 bytes aleatorios en hexadecimal
challenge = sha256(salt . number)
expires   = ahora + minutos configurados
firma     = hmac_sha256(secreto, salt|challenge|maxnumber|ts|expires|formulario|huella)
```

El navegador prueba 0, 1, 2... hasta que `sha256(salt . n) === challenge`. Con `maxnumber = 30000` son 15.000 intentos de media: unos 50 ms con la implementación de SHA-256 en JavaScript puro que lleva el módulo.

Por qué SHA-256 propio y no `crypto.subtle`: `crypto.subtle` solo existe en contexto seguro (no en `http://`) y es asíncrono, así que obligaría a encadenar 30.000 promesas. La versión síncrona en JavaScript puro es más rápida aquí y funciona en todas partes. El bucle va troceado en bloques de 3.000 con `setTimeout` para no congelar la pantalla.

El servidor comprueba (`EcomBlockbotChallenge::verify`), en este orden:

1. La firma cuadra (`hash_equals`) → la ficha la emitimos nosotros y nadie la ha tocado.
2. `expires` no ha pasado y `ts` no es más viejo que el máximo.
3. `sha256(salt . number) === challenge` → el trabajo está hecho de verdad.
4. El `salt` no está en `ecom_blockbot_nonce` → no es un reenvío. Se inserta.
5. `ahora - ts >= segundos mínimos` → no se ha enviado al instante.

La **huella** ata la ficha al navegador y a la subred **/24**, no a la IP exacta: en móvil la IP cambia sola y atarla daría falsos positivos.

El secreto (`ECOMBB_SECRET`, 64 caracteres) se genera en la instalación con `random_bytes` y es **global** a todas las tiendas: la ficha tiene que seguir valiendo si el visitante salta de tienda.

4. Motor de puntuación

`EcomBlockbotDetector::analyze($form, $data)` devuelve `array('score' => int, 'reasons' => array, 'block' => bool, 'action' => string)`.

Clave	Puntos	Bloquea solo	Qué comprueba
<code>honeypot</code>	100	sí	Un campo trampa llega relleno
<code>no_token</code>	100	sí	No llega la ficha → no ha corrido JavaScript
<code>bad_token</code>	100	sí	Firma inválida, manipulada o de otro formulario
<code>pow_fail</code>	100	sí	El número no resuelve el reto
<code>replay</code>	100	sí	Ficha ya usada
<code>expired</code>	100	sí	Ficha caducada
<code>banned</code>	100	sí	IP en lista negra o bloqueo automático vigente
<code>rate_hour</code> / <code>rate_day</code>	100	sí	Se pasa del límite de envíos por IP
<code>webdriver</code>	80	no	<code>navigator.webdriver === true</code>
<code>too_fast</code>	70	no	Menos de N segundos entre la ficha y el envío
<code>bad_ua</code>	70	no	Agente de usuario vacío, corto o de la lista de robots
<code>disposable</code>	60	no	Dominio de correo de usar y tirar
<code>no_mx</code>	55	no	El dominio no tiene ni MX ni A
<code>links</code>	50	no	Más enlaces de los permitidos
<code>alphabet</code>	45	no	Más de un tercio del texto en alfabeto marcado
<code>bad_origin</code>	45	no	<code>Origin / Referer</code> ausente o de otro dominio
<code>no_interaction</code>	40	no	Cero eventos de ratón, teclado o toque
<code>keyword</code>	40	no	Palabra de la lista de vigilancia
<code>name_junk</code>	30	no	Nombre que es un correo, una URL o una ristra sin vocales
<code>no_accept_lang</code>	25	no	Sin cabecera <code>Accept-Language</code>

Umbral de fábrica **100**: de serie solo bloquean solos los motivos que no tienen falsos positivos. Los demás suman y hacen falta dos.

Cortocircuito: en cuanto la puntuación llega al umbral se dejan de ejecutar las comprobaciones caras (DNS, contenido).

La comprobación de DNS se desactiva sola si el servidor no tiene DNS saliente: antes de usarla se hace una sonda contra un dominio conocido. Si no, todos los correos parecerían falsos.

5. Ficheros

```
ecom_blockbot/
ecom_blockbot.php           clase Ecom_Blockbot (espacio de nombres global)
index.php  logo.png
DOC-tecnico.md  DOC-desarrollo.md  DOC-usuario.md
classes/
  EcomBlockbotChallenge.php    emitir y verificar la ficha + prueba de trabajo
  EcomBlockbotDetector.php     motor de puntuación y decisión
  EcomBlockbotHeuristics.php   agente de usuario, cabeceras, correo, contenido
  EcomBlockbotStore.php        base de datos: registro, IP, fichas, purga
  EcomBlockbotLogger.php       depuración a logs/*.log
controllers/front/challenge.php endpoint de la ficha
controllers/admin/AdminEcomBlockbotLogsController.php
views/css/admin.css
views/js/ecomblockbot.js      SHA-256, inyección, telemetría, resolución
views/templates/admin/dashboard.tpl resumen + autocomprobación
views/templates/admin/ip_rules.tpl listas blanca y negra
translations/es-ES/ModulesEcomblockbotAdmin.es-ES.xlf
translations/es-ES/ModulesEcomblockbotShop.es-ES.xlf
sql/schema.sql                referencia (las tablas las crea la clase Store)
data/disposable.txt            569 dominios de correo desechable
logs/ (+ .htaccess que impide servir los .log)
```

`index.php` en todas las carpetas, recursivo. Cabecera de copyright en todos los ficheros.

6. Base de datos

```
ecom_blockbot_log  id_log, id_shop, date_add, ip, ip_long, ua, form,
                  verdict, score, reasons, email, uri
                  KEY(date_add) KEY(ip,date_add) KEY(verdict,date_add)

ecom_blockbot_ip   id_rule, id_shop, pattern, ip_start BIGINT, ip_end BIGINT,
                  type (0=negra, 1=blanca), auto, reason, hits, date_add, date_end
                  UNIQUE(pattern,type) KEY(ip_start,ip_end) KEY(type)

ecom_blockbot_nonce salt CHAR(32) PRIMARY KEY, date_add
```

La columna se llama **verdict** y no **action** a propósito: **action** es el nombre de un parámetro que usa el propio `AdminController` y da más de un quebradero de cabeza.

Rangos de IP: se admite IP exacta, CIDR IPv4 (`1.2.3.0/24`) y comodín (`1.2.3.*`). Se guardan resueltos a `ip_start / ip_end` para que la consulta sea un `BETWEEN` con índice. Las IPv6 se comparan por igualdad exacta.

Purga automática: el registro se limpia por los días configurados; las fichas usadas, a los 2 días; los bloqueos automáticos caducados, a los 7. Se dispara 1 de cada 50 peticiones al endpoint, al abrir la configuración y con el botón de la pantalla del registro.

Las tablas se crean en `EcomBlockbotStore::createTables()`, invocada desde `install()` y desde `addnewfeatures()`. **No hay ficheros** `upgrade/`.

7. Configuración y multitienda

Claves con prefijo `ECOMBB_`.

- **Globales** (`getGlobalValue / updateGlobalValue`): `ECOMBB_SECRET` (clave de firma), `ECOMBB_PREFIX` (prefijo de los campos ocultos) y `ECOMBB_HP` (nombres de los campos trampa). Tienen que ser comunes porque la ficha debe valer si el visitante cambia de tienda.
- **Por tienda** (`Configuration::get / updateValue`): todo lo demás.

Cuando `Shop::isFeatureActive()` es verdadero, la pantalla lo avisa explícitamente.

`ECOMBB_MSG` (el mensaje que ve el visitante bloqueado) es **multiidioma**. Si se deja vacío, se usa la traducción del módulo en el idioma del visitante.

Los nombres de los campos

El prefijo (`bb` + 6 caracteres aleatorios) y los dos campos trampa (`website_a3f1`, `confirm_email_a3f1` ...) se sortean **en la instalación**. El sufijo aleatorio no es para despistar al robot —que rellena todo lo que ve, se llame como se llame— sino para que **no puedan chocar nunca con un campo de verdad del tema o de otro módulo**, que sería un cliente legítimo bloqueado por error.

8. El endpoint de la ficha

`controllers/front/challenge.php` → `index.php?`

`fc=module&module=eecom_blockbot&controller=challenge&form=<clave>`

- Responde JSON con **las tres cabeceras anti-caché** (`Cache-Control: no-store...`, `Pragma`, `Expires`) y **ningún Content-Type propio**: con `application/json`, jQuery convertiría la respuesta y el `JSON.parse` del JavaScript reventaría. Se consume con `fetch` + `JSON.parse` del texto, nunca con jQuery.
- Se pide con `cache: 'no-store'` y un parámetro aleatorio, para tapar también proxies y CDN.

Autocomprobación: la pantalla de configuración llama a este endpoint desde el navegador y enseña en verde o en rojo si responde. Es la comprobación que caza el fallo que ninguna revisión estática ve: si el endpoint no contesta (URL amigables mal, `.htaccess`, módulo desactivado), **ningún visitante podría enviar un formulario protegido**.

9. Pruebas

Están fuera del módulo, en `_specs/ecom_blockbot/001-bloqueo-bots-formularios/herramientas/`:

Guion	Qué prueba	Comprobaciones
<code>prueba_blockbot.php</code>	Ficha, prueba de trabajo, rangos de IP, heurísticas. Sin PrestaShop	49
<code>prueba_interop.php</code>	El SHA-256 que se sirve al navegador resuelve el reto que emite PHP	2
<code>prueba_tienda.py</code>	Tienda real: robot bloqueado, persona pasa, campo trampa, navegador automatizado	22
<code>prueba_config.py</code>	Pantalla de configuración: pestañas dentro del formulario, guardado, botones	21
<code>prueba_registro.py</code>	Pantalla del registro: todos los botones, filtros, borrados	17

Además: `extraer_cadenas.py` compara las cadenas del código con los `resname` de los `.xlf` y `generar_xlf.py` los regenera.

10. Límites conocidos

- **Un visitante sin JavaScript no puede enviar** los formularios protegidos. Es intencionado (es lo que corta a `curl`, a Python y a todos los robots de línea de órdenes) y se puede apagar con «Exigir una ficha firmada».
- Un atacante decidido con un navegador real automatizado y bien disfrazado puede pasar. El objetivo no es la impenetrabilidad: es que el spam masivo deje de salir a cuenta.
- La detección por comportamiento se puede falsificar. Suma puntos, nunca bloquea sola.
- No cubre los comentarios de producto ni ningún formulario de módulos de terceros que no use uno de los cinco campos de envío vigilados.

Ecom Blockbot — guía para quien toque el código

Versión: 1.0.0 · **Autor:** Ecom Experts · ecomyseo@gmail.com · AFL-3.0

Este documento es para el desarrollador que va a modificar el módulo. Lo que hace y cómo está montado está en `DOC-tecnico.md` ; cómo se usa, en `DOC-usuario.md` .

Reglas de la casa que este módulo cumple

- Estructura **legacy**: sin `src/` , sin `config/services.yml` , sin `composer.json` , sin `vendor/` , sin `upgrade/` . Clase principal `Ecom_Blockbot` en el espacio de nombres global.
- Clases auxiliares en `classes/` , cargadas con `require_once` desde la clase principal.
- Migraciones y altas de hooks en `addnewfeatures()` , envuelto en `try{}` , invocado desde `install()` y desde `getContent()` . Nunca ficheros de actualización.
- Configuración: **UN solo HelperForm con pestañas y un único botón de guardar.**
- `isUsingNewTranslationSystem()` a `true` . Todas las cadenas con `trans('...', array(), 'Modules.Ecomblockbot.Admin|Shop')` o `{1 s='...' d='...'}` . **Ni un mod=** : el sistema antiguo no lee los `.xlf` .
- `index.php` en todas las carpetas, recursivo. Cabecera de copyright en todos los ficheros.
- Nunca `foreach ((array) $x)` sobre algo que puede ser `false` : siempre `is_array($x) ? $x : array()` .

Las cinco trampas que ya costaron tiempo aquí

1. `Module::trans()` es `protected`

En `classes/module/Module.php:3396` (8.2.7) y `:3499` (9.1.1). Desde el controlador de administración o desde una clase de `classes/` , `$this->module->trans(...)` lanza `Call to protected method` y **la pantalla se cae con un 500 en blanco**.

Se arregla ampliando la visibilidad en la clase del módulo, con la firma exacta del núcleo:

```
public function trans($id, array $parameters = array(), $domain = null, $locale = null)
{
    return parent::trans($id, $parameters, $domain, $locale);
}
```

Renombrar el método (`translate()`) **no vale**: el extractor de cadenas deja de verlas.

2. Las claves de las pestañas se convierten en `id` del DOM

`js/admin.js` (líneas 674-693) hace `$('#<div id="'+clave+'">')` y luego `$(campos).appendTo('#'+clave)`. `unique_field_id` prefija el `fieldset`, **no la pestaña**.

Si la clave choca con un `id` que ya existe en el back-office —y **content existe**, es el contenedor principal— jQuery mueve esos campos **fuera del `<form>`**: la pantalla sale descuadrada y **esa pestaña entera deja de guardarse**, diciendo «Configuración guardada». Ni un error, ni un aviso, ni nada en el registro.

Por eso las pestañas se llaman `bbgeneral`, `bbforms`, `bbchecks`, `bbcontent`, `bblimits`. La prueba `prueba_config.py` comprueba que las cinco siguen dentro del formulario.

3. `actionSubmitAccountBefore` tiene que devolver `true` SIEMPRE

`RegistrationController.php:56` hace `array_reduce(Hook::exec(...), fn($carry, $item) => $carry && $item, true)`. Si el hook devuelve `null` (por ejemplo por un `return;` a secas), **se bloquean todos los registros de la tienda**. Cada camino del método tiene que terminar en `return true;` o `return false;`.

4. El endpoint AJAX, sin `Content-Type`

Las **tres** cabeceras anti-caché (`Cache-Control: no-store...`, `Pragma`, `Expires`) y **ninguna más**. Poner `Content-Type: application/json` hace que jQuery entregue la respuesta ya convertida en objeto, y el `JSON.parse` de encima revienta con `"[object Object]" is not valid JSON : 200` en el servidor, nada en el registro, y la funcionalidad deja de hacer nada.

5. El registro no es un `ObjectModel` a propósito

Es una tabla plana de solo lectura. Como no hay `$this->className`, `AdminController` no puede borrar solo (haría `new $this->className($id)`), así que `deleteecom_blockbot_log` y `submitBulkdeleteecom_blockbot_log` se manejan a mano en `postProcess()` y se quitan del POST para que el padre no lo intente.

Puntos de extensión

Proteger otro formulario

1. Añadir la constante en `EcomBlockbotDetector` (`FORM_LOQUESEA`).
2. Añadir la clave a `config()`, a `isFormEnabled()`, a `collect()` (qué campos del POST hay que analizar) y a la lista `$allowed` del controlador `challenge`.
3. Añadir el interruptor `ECOMBB_F_LOQUESEA` en `defaults()`, en `renderForm()` (pestaña `bbforms`) y en `saveConfig()`.
4. Añadir la entrada al `KEYMAP` de `views/js/ecomblockbot.js` con el `name` del campo de envío.
5. Cortar el envío: si el formulario expone un hook propio, usarlo; si no, en `hookActionDispatcher`, quitando la clave del POST y llamando a `pushError()`.
6. Regenerar las traducciones y añadir la comprobación a `prueba_tienda.py`.

Cambiar los pesos

Están en `EcomBlockbotDetector::weights()`. Cualquier motivo con 100 bloquea solo con el umbral de fábrica. **Antes de subir un peso por encima de 99, hay que estar seguro de que esa comprobación no da falsos positivos.**

Añadir una comprobación

Método estático en `EcomBlockbotHeuristics` (sin estado, fácil de probar), llamada en `analyze()` **detrás de un `if ($result['score'] < $cfg['threshold'])`** para respetar el cortocircuito, entrada en `weights()`, etiqueta en `reasonLabel()` e interruptor propio en la pestaña de comprobaciones.

Tocar el JavaScript

`views/js/ecomblockbot.js` va en ES5 a propósito (`var`, sin flechas, sin promesas propias), para no depender del navegador del cliente. El SHA-256 está verificado contra `crypto.createHash` de Node en 5 casos y contra PHP en `prueba_interop.php`: **si se toca, hay que volver a pasar esa prueba.**

Cómo se prueba

Todo está fuera del módulo, en `z:\modulos ps mios finalizados y ok\modulos IA\specs\ecom_blockbot\001-bloqueo-bots-formularios\herramientas\`:

<code>php prueba_blockbot.php ../../ecom_blockbot</code>	49 comprobaciones, sin PrestaShop
<code>php prueba_interop.php ../../ecom_blockbot</code>	PHP <-> JavaScript de la prueba de trabajo
<code>python prueba_tienda.py</code>	22, contra la tienda 8.2b
<code>python prueba_config.py</code>	21, pantalla de configuracion
<code>python prueba_registro.py</code>	17, pantalla del registro
<code>python extraer_cadenas.py <carpeta></code>	cadenas del código contra los .xlf
<code>python generar_xlf.py <carpeta></code>	regenera los .xlf en castellano

Detalles que no son obvios:

- `prueba_tienda.py` **usa las URL amigables** (`/registro`, `/contactenos`). Con `index.php?controller=...` la tienda devuelve una redirección, `requests` la sigue con GET, el POST nunca llega y la prueba pasaría **sin probar nada**.
- **Playwright tal cual se detecta como robot**: declara `HeadlessChrome` y enciende `navigator.webdriver`. Para simular a una persona hay que darle un `user_agent` normal y un `add_init_script` que apague `navigator.webdriver`. Que sin ese disfraz lo bloquee es en sí una de las pruebas.
- En el back-office, `wait_for_load_state('networkidle')` **no es fiable** (hay peticiones periódicas): hay que usar `expect_navigation()` y esperar a un selector concreto.
- Playwright **descarta** los `confirm()` por omisión: sin `page.on('dialog', accept)` los botones con confirmación no llegan a enviar y la prueba falla sin motivo aparente.
- El botón «Eliminar» de una lista de PrestaShop **no usa `confirm()`**: abre `jConfirm` (`jquery.alerts`), cuyo botón de aceptar es `#popup_ok`.

- Cada mitad de `prueba_tienda.py` **vacía las tres tablas** antes de empezar: si no, el bloqueo automático deja banda la IP local y la mitad siguiente falla por eso.

Traducciones

Las cadenas fuente van **en inglés** (son la clave con la que PrestaShop busca en el `.xlf`) y el `<target>` en castellano correcto, con tildes y eñes.

Los `.xlf` **no se editan a mano**: se añade el par inglés → castellano al diccionario de `generar_xlf.py` y se regenera. `extraer_cadenas.py` devuelve código de salida 1 si falta alguna, así que sirve para un enganche previo al commit.

La ruta importa: en ejecución PrestaShop solo lee `translations/<locale>/Modules<Nombre><Dominio>.<locale>.xlf`. Un `.xlf` monolítico en la raíz solo lo ve el editor de traducciones del back-office.

Depuración

Interruptor «Modo de depuración» en la pestaña General. Escribe `logs/blockbot-AAAA-MM-DD.log` con la ficha emitida, el resultado de cada análisis y los bloqueos automáticos. Rota a los 2 MB. El `.htaccess` de la carpeta impide servir los `.log` por web. La pantalla lista los ficheros y trae un botón para borrarlos.